

Algoritmi de clasificare

Cuprins

Descrierea problemei	2
Setul de date ales	2
Prelucrarea datelor	3
Parametrii statistici de evaluare	3
Perceptronul (cod și experimente)	3
Naive Bayes (cod și experimente)	6
KNearest Neighbors (cod și experimente)	9
Support Vector Machine (cod și experimente)	12
Arbori de decizie (cod și experimente)	15
Regulile de decizie	18
Tabel centralizator comparativ	18

Listă figuri

Fig. 1 - Matricea de confuzie exp. 1 Perceptron	5
Fig. 2 - Valori reale vs predicție exp. 1 Perceptron	6
Fig. 3 - Matricea de confuzie exp. 1 Naive Bayes	8
Fig. 4 - Valori reale vs predicție exp. 1 Naive Bayes	8
Fig. 5 - Matricea de confuzie exp. 1 KNearest Neighbors	11
Fig. 6 - Valori reale vs predicție exp. 1 KNearest Neighbors	11
Fig. 7 - Matricea de confuzie exp. 1 Support Vector Machine	14
Fig. 8 - Valori reale vs predicție exp. 1 Support Vector Machine	14
Fig. 9 – Arborele de decizie în format text exp. 1 Arbori de decizie	17
Fig. 10 - Arborele de decizie reprezentat grafic exp. 1 Arbori de decizie	17

Descrierea problemei

Problema constă în analizarea impactului utilizării rețelelor de socializare și a stilului de viață asupra sănătății mentale a adolescenților. Obiectivul este de a crea un model de învățare automată capabil să clasifice dacă un adolescent prezintă sau nu simptome de depresie (*depression_label*), pe baza unor atribute precum timpul petrecut în fața ecranelor, calitatea somnului și activitatea fizică.

Setul de date ales

Sursa: [Kaggle \(Teenager Mental Health Dataset\)](#).

Număr de înregistrări: 1200.

Variabilă scop aleasă: depression_label.

Variabile utilizate pentru clasificare: age, gender, daily_social_media_hours, platform_usage, sleep_hours, screen_time_before_sleep, academic_performance, physical_activity, social_interaction_level.

Parametrii din acest fișierul *Teen_Mental_Health_Dataset.csv* sunt prezentați în tabelul de mai jos.

Nume parametru	Descriere parametru	Tip valoare	Interval valoare	Unit. măsură
age	Vârsta adolescentului	Număr natural	13-19	Ani
gender	Sexul	Șir de caractere	male/female	-
daily_social_media_hours	Timpul zilnic petrecut pe rețelele sociale	Număr întreg	1.0-8.0	Ore
platform_usage	Platforma utilizată	Șir de caractere	Instagram/TikTok/Both	-
sleep_hours	Numărul orelor de somn	Număr întreg	4.0-9.0	Ore
screen_time_before_sleep	Numărul orelor petrecute la ecran înainte de somn	Număr întreg	0.5-3.0	Ore
academic_performance	Performanța academică	Număr întreg	2.0-4.0	Scor
physical_activity	Activitatea fizică	Număr întreg	0.0-4.0	Ore

social_interaction_level	Nivelul de interacțiune socială	Șir de caractere	low/medium/high	Scară
stress_level	Nivelul de stres	Număr întreg	1-10	Scară
anxiety_level	Nivelul de anxietate	Număr natural	1-10	Scară
addiction_level	Nivelul de dependență	Număr natural	1-10	Scară
depression_label	Diagnosticat cu depresie	Număr natural	0 (Nu), 1 (Da)	-

Prelucrarea datelor

În setul de date nu există valori lipsă. Există totuși atributele *gender*, *platform_usage* și *social_interaction_level* ce trebuie transformate din text în valori numerice (**Encoding**).

Deoarece avem intervale diferite (adică vârsta este între 13-19, orele petrecute pe social media sunt între 1-8, media e 2-4) trebuie să aplicăm o metodă de standardizare a datelor (**StandardScaler**).

S-au **eliminat atributele**: *stress_level*, *anxiety_level*, *addiction_level*.

Parametrii statistici de evaluare

Acuratețea (Accuracy): Reprezintă raportul dintre numărul de predicții corecte și numărul total de predicții.

Matricea de confuzie (Confusion Matrix): Tabel ce arată performanța algoritmului.

- True Positives (TP): Diagnosticați corect cu depresie.
- True Negatives (TN): Identificați corect ca fiind sănătoși.
- False Positives (FP): Sănătoși clasificați ca având depresie.
- False Negatives (FN): Bolnavi clasificați ca sănătoși.

Raportul de clasificare (Classification Report): Include:

- Precision: Câți din cei marcați cu depresie chiar suferă de boală.
- Recall: Câți dintre cei care au depresie au fost găsiți de model.
- F1-Score: Media armonică între precizie și recall.

Perceptronul (cod și experimente)

Codul pentru acest model și primul test vor fi lăsate mai jos.

```
import pandas as pd
```

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import Perceptron
from sklearn.model_selection import KFold, cross_val_score, cross_val_predict
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report, ConfusionMatrixDisplay

# incarcam datele
df = pd.read_csv('data/Teen_Mental_Health_Dataset.csv')

# eliminam attributele stress_level, anxiety_level si addiction_level
df = df.drop(['stress_level', 'anxiety_level', 'addiction_level'], axis=1)

# transformam variabile text in numerice
def encode_text(df, columns):
    le = LabelEncoder()
    for col in columns:
        df[col] = le.fit_transform(df[col])
    return df

categoriale = ['gender', 'platform_usage', 'social_interaction_level']
df = encode_text(df, categoriale)

# separare X (atribute) si y (scop)
X = df.drop('depression_label', axis=1).values.astype(np.float32)
y = df['depression_label'].values.astype(np.float32)

# standardizam datele
scaler = StandardScaler()
X_std = scaler.fit_transform(X)

# functie pentru rularea experimentului
def experiment_perceptron(k_val, eta, max_it):
    print(f"\n{'='*30}")
    print(f"EXPERIMENT: K={k_val}, eta0={eta}, max_iter={max_it}")

    ppn = Perceptron(eta0=eta, max_iter=max_it, random_state=42)
    kf = KFold(n_splits=k_val, shuffle=True, random_state=42)

    # genera, predictiile prin Cross-Validation (K-Fold)
    y_pred = cross_val_predict(ppn, X_std, y, cv=kf)

    # calculam si afisam acuratetea
    acc = accuracy_score(y, y_pred)
    print(f"ACURATEȚE: {acc:.4f}")

    # afisam raportul de clasificare (precision, recall, F1-Score)
    print("\nRAPORT DE CLASIFICARE:")
    print(classification_report(y, y_pred))

    # calculam matricea de confuzie
    cm = confusion_matrix(y, y_pred)
    print("MATRICEA DE CONFUZIE:")
    print(cm)

    # partea de rapoarte

```

```

fig, ax = plt.subplots(1, 2, figsize=(12, 5))

# matricea de confuzie (Heatmap)
ConfusionMatrixDisplay(confusion_matrix=cm).plot(ax=ax[0], cmap='Blues')
ax[0].set_title(f"Matrice Confuzie (K={k_val})")

# comparare valori reale vs predicții
# luam doar primele 50 de instanțe
t = pd.DataFrame({'real': y[:50], 'predit': y_pred[:50]})
ax[1].plot(t['real'].tolist(), label='Real', marker='o', linestyle='')
ax[1].plot(t['predit'].tolist(), label='Predit', marker='x',
linestyle='')
ax[1].set_title("Real vs Predictie (esantion 50)")
ax[1].legend()

plt.tight_layout()
plt.show()

# experimentul 1
experiment_perceptron(k_val=5, eta=0.01, max_it=100)

```

În continuare va fi prezentat mai jos tabelul centralizator în care vor fi scrise următoarele valori: k (Folds), eta0, max_iter, early_stopping, acuratețea medie și precizia (pentru valori de 0 și valori de 1). Totodată vor fi puse mai jos 2 poze cu matricea de confuzie, respectiv compararea valorilor reale cu predicțiile.

Nr. exp.	K (Folds)	eta0	max_iter	early_stopping	Acuratețe medie	Precizie (0 / 1)
1	5	0.01	100	False	0.9542	0.98 / 0.1
2	5	0.1	100	False	0.9550	0.97 / 0.0
3	5	0.01	100	False	0.9542	0.98 / 0.1
4	5	0.01	1000	True	0.9575	0.97 / 0.0
5	10	0.01	100	False	0.9450	0.98 / 0.05

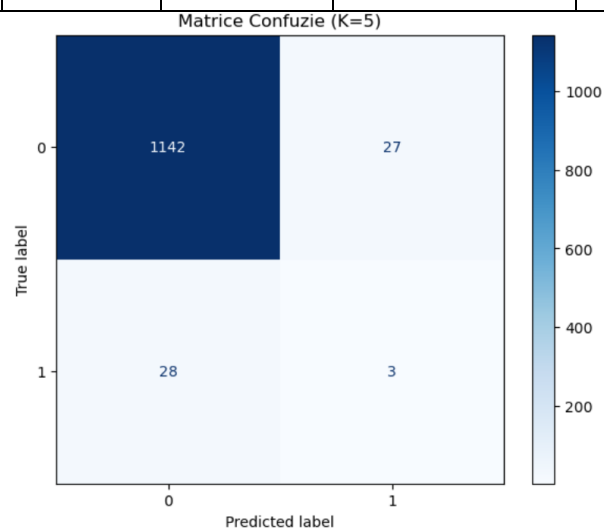


Fig. 1 - Matricea de confuzie exp. 1 Perceptron

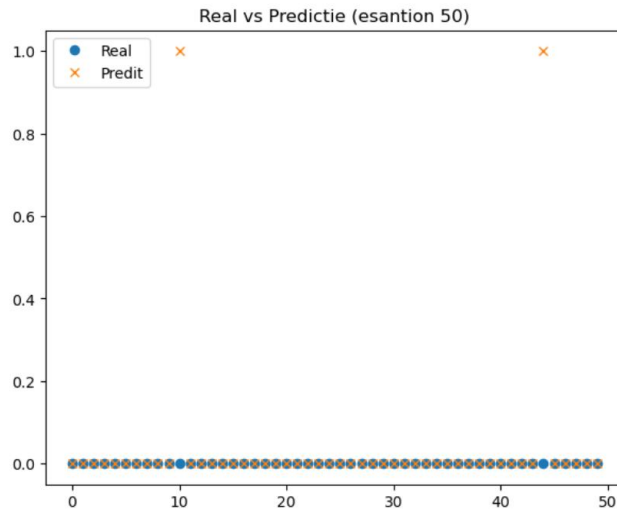


Fig. 2 - Valori reale vs predicție exp. 1 Perceptron

Naive Bayes (cod și experimente)

Codul pentru acest model și primul test vor fi lăsate mai jos.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import KFold, cross_val_predict
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report, ConfusionMatrixDisplay

# eliminam formatul stiintific pentru probabilitati
np.set_printoptions(suppress=True)

# incarcam datele
df = pd.read_csv('data/Teen_Mental_Health_Dataset.csv')

# eliminam attributele stress_level, anxiety_level si addiction_level
df = df.drop(['stress_level', 'anxiety_level', 'addiction_level'], axis=1)

# transformam variabile text in numerice
def encode_text(df, columns):
    le = LabelEncoder()
    for col in columns:
        df[col] = le.fit_transform(df[col])
    return df

categoriale = ['gender', 'platform_usage', 'social_interaction_level']
df = encode_text(df, categoriale)

# separare X (atribute) si y (scop)
X = df.drop('depression_label', axis=1).values.astype(np.float32)
y = df['depression_label'].values.astype(np.float32)
```

```

# standardizam datele
scaler = StandardScaler()
X_std = scaler.fit_transform(X)

# functie pentru rularea experimentului Naive Bayes
def experiment_naive_bayes(k_val, nume_exp):
    print(f"\n\n--- {nume_exp} ---")

    gnb = GaussianNB()
    kf = KFold(n_splits=k_val, shuffle=True, random_state=42)

    # predictiile modelului prin k-fold
    y_pred = cross_val_predict(gnb, X_std, y, cv=kf)

    # calcularea probabilitatilor
    y_proba = cross_val_predict(gnb, X_std, y, cv=kf,
method='predict_proba')

    # afisarea claselor si a probabilitatilor pentru primele instante
    # (antrenam modelul pe tot setul doar pentru a vedea clasele)
    gnb.fit(X_std, y)
    print('\nClasele:', gnb.classes_)
    print('Probabilitatile pentru primele 5 instante test (%):\n',
y_proba[:5] * 100)

    # numararea instantelor clasificate gresit
    print("\nInstante clasificate gresit dintr-un total de %d inregistrari :
%d" % (len(y), (y != y_pred).sum()))

    # acuratetea de clasificare
    print("Acuratetea de clasificare = % 5.2f" % accuracy_score(y, y_pred))

    # afisare raport de clasificare
    print("\nRaportul de clasificare:")
    print(classification_report(y, y_pred))

    # matricea de confuzie
    cm = confusion_matrix(y, y_pred)
    print('Matricea de confuzie:\n', cm)

    # reprezentare grafica
    fig, ax = plt.subplots(1, 2, figsize=(12, 5))

    # matricea de confuzie grafica
    ConfusionMatrixDisplay(confusion_matrix=cm).plot(ax=ax[0],
cmap='Greens')
    ax[0].set_title(f"Matrice Confuzie {nume_exp}")

    # comparare valori reale vs predictii (esantion 50 instante)
    t = pd.DataFrame({'real': y[:50], 'predit': y_pred[:50]})
    ax[1].plot(t['real'].tolist(), label='Real', marker='o', linestyle='')
    ax[1].plot(t['predit'].tolist(), label='Predit', marker='x',
linestyle='')
    ax[1].set_title("Real vs Predictie NB")
    ax[1].legend()

```



```
plt.tight_layout()
plt.show()

# experiment 1
experiment_naive_bayes(k_val=5, nume_exp="Experiment_1")
```

În continuare va fi prezentat mai jos tabelul centralizator în care vor fi scrise următoarele valori: k (Folds), instanțe clasificate greșit, acuratețea medie și precizia (pentru valori de 0 și valori de 1). Totodată vor fi puse mai jos 2 poze cu matricea de confuzie, respectiv compararea valorilor reale cu predicțiile.

Nr. exp.	K (Folds)	Instanțe clasificate greșit	Acuratețe medie	Precizie (0 / 1)
1	5	37	0.97	0.97 / 0.0
2	10	33	0.97	0.97 / 0.0
3	3	38	0.97	0.97 / 0.0

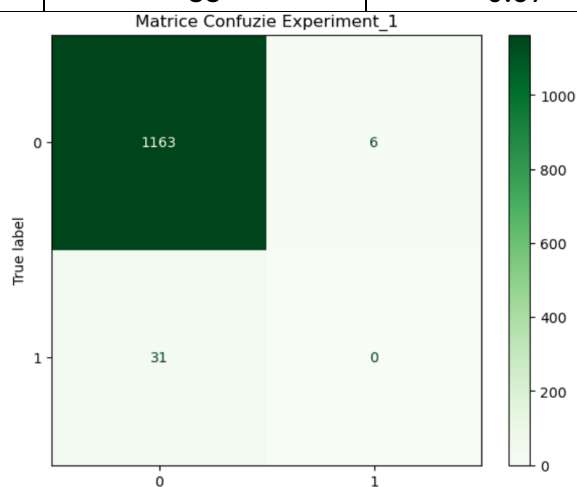


Fig. 3 - Matricea de confuzie exp. 1 Naive Bayes

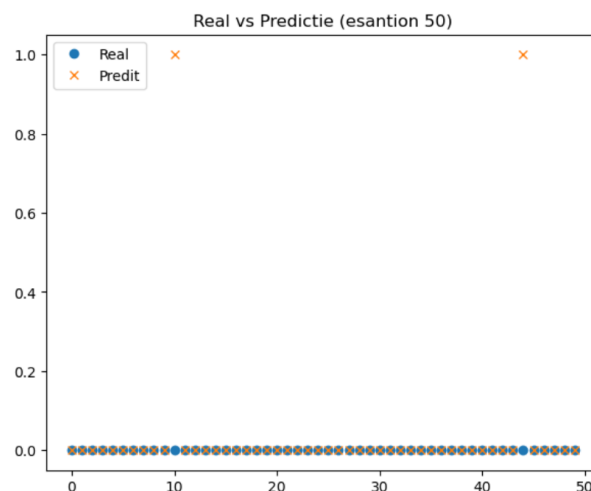


Fig. 4 - Valori reale vs predicție exp. 1 Naive Bayes

KNearest Neighbors (cod și experimente)

Codul pentru acest model și primul test vor fi lăsate mai jos.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import KFold, cross_val_predict
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report, ConfusionMatrixDisplay

# eliminam formatul stiintific pentru probabilitati
np.set_printoptions(suppress=True)

# incarcam datele
df = pd.read_csv('data/Teen_Mental_Health_Dataset.csv')

# eliminam attributele stress_level, anxiety_level si addiction_level
df = df.drop(['stress_level', 'anxiety_level', 'addiction_level'], axis=1)

# transformam variabile text in numerice
def encode_text(df, columns):
    le = LabelEncoder()
    for col in columns:
        df[col] = le.fit_transform(df[col])
    return df

categoriale = ['gender', 'platform_usage', 'social_interaction_level']
df = encode_text(df, categoriale)

# separare X (atribute) si y (scop)
X = df.drop('depression_label', axis=1).values.astype(np.float32)
y = df['depression_label'].values.astype(np.float32)

# standardizam datele
scaler = StandardScaler()
X_std = scaler.fit_transform(X)

# functie pentru rularea experimentului KNN
def experiment_knn(k_fold_val, n_vecini, p_param, alg_val, nume_exp):
    print(f"\n\n--- {nume_exp} ---")
    print(f"Parametri: Vecini={n_vecini}, p={p_param} (Minkowski),
    Algoritm={alg_val}")

    # construirea modelului KNN
    # p=2 inseamna distanta Euclidiană, p=1 inseamna distanta Manhattan
    knn = KNeighborsClassifier(n_neighbors=n_vecini, p=p_param,
    algorithm=alg_val, metric='minkowski')
    kf = KFold(n_splits=k_fold_val, shuffle=True, random_state=42)

    # predictiile modelului prin k-fold
    y_pred = cross_val_predict(knn, X_std, y, cv=kf)

    # calcularea probabilitatilor
```

```

y_proba = cross_val_predict(knn, X_std, y, cv=kf,
method='predict_proba')

# afisarea claselor si a probabilitatilor pentru primele instante
knn.fit(X_std, y)
print('\nClasele:', knn.classes_)
print('Probabilitatile ptr. primele 5 instante test (%):\n', y_proba[:5]
* 100)

# numararea instantelor clasificate gresit
print("\nInstante clasificate gresit dintr-un total de %d inregistrari :
%d" % (len(y), (y != y_pred).sum()))

# acuratetea de clasificare
print("Acuratetea de clasificare = % 5.2f" % accuracy_score(y, y_pred))

# afisare raport de clasificare complet
print("\nRaportul de clasificare:")
print(classification_report(y, y_pred))

# matricea de confuzie numeric
cm = confusion_matrix(y, y_pred)
print('Matricea de confuzie:\n', cm)

# reprezentare grafica
fig, ax = plt.subplots(1, 2, figsize=(12, 5))

# matricea de confuzie grafica
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
display_labels=knn.classes_)
disp.plot(ax=ax[0], cmap='YlOrBr')
ax[0].set_title(f"Matrice Confuzie {nume_exp}")

# comparare valori reale vs predictii (esantion 50 instante)
t = pd.DataFrame({'real': y[:50], 'predit': y_pred[:50]})
ax[1].plot(t['real'].tolist(), label='Real', marker='o', linestyle='')
ax[1].plot(t['predit'].tolist(), label='Predit', marker='x',
linestyle='')
ax[1].set_title("Real vs Predictie KNN")
ax[1].legend()

plt.tight_layout()
plt.show()

# experiment 1
experiment_knn(k_fold_val=5, n_vecini=5, p_param=2, alg_val='auto',
nume_exp="Exp KNN 1")

```

În continuare va fi prezentat mai jos tabelul centralizator în care vor fi scrise următoarele valori: k (Folds), n_neighbors, distanța, instanțe clasificate greșit, acuratețea medie și precizia (pentru valori de 0 și valori de 1). Totodată vor fi puse mai jos 2 poze cu matricea de confuzie, respectiv compararea valorilor reale cu predicțiile.

Nr. exp.	K (Folds)	n_neighbors	Distanța	Instanțe greșite	Acuratețe medie	Precizie (0 / 1)
1	5	5	2	31	0.97	0.97 / 0.0
2	5	1	2	51	0.96	0.98 / 0.21
3	5	5	1	32	0.97	0.97 / 0.0
4	5	15	2	31	0.97	0.97 / 0.0
5	10	7	2	31	0.97	0.97 / 0.0

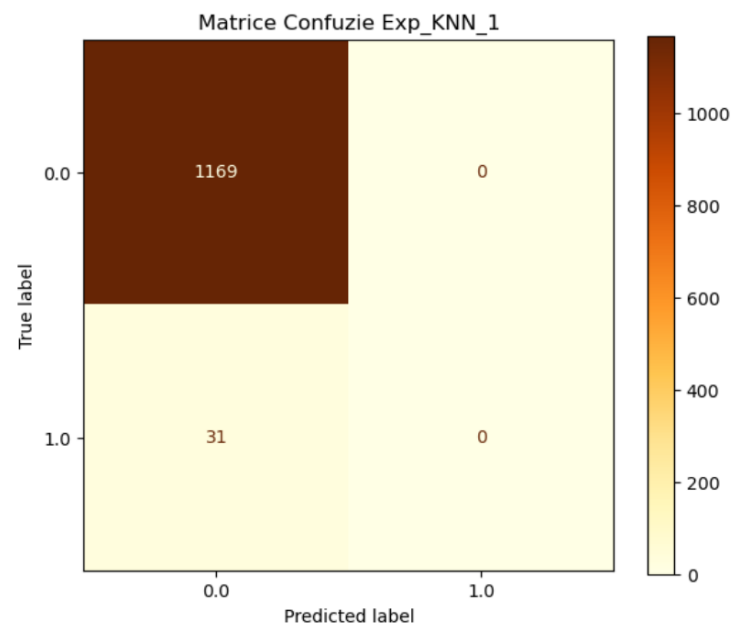


Fig. 5 - Matricea de confuzie exp. 1 KNearest Neighbors

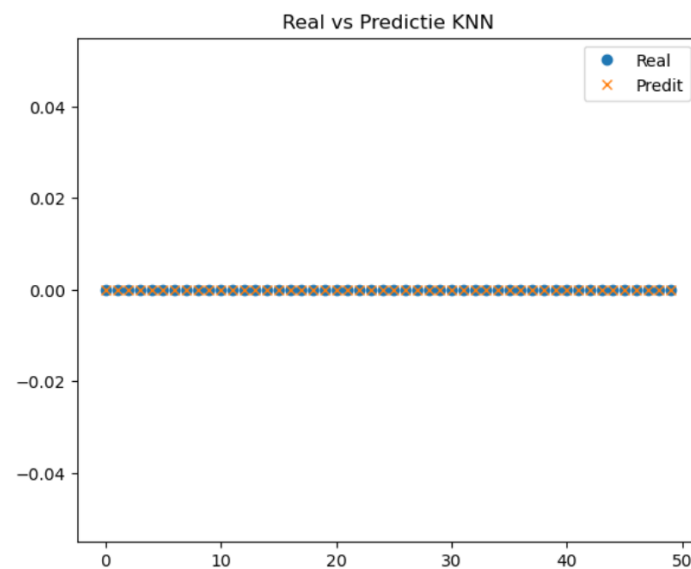


Fig. 6 - Valori reale vs predicție exp. 1 KNearest Neighbors

Support Vector Machine (cod și experimente)

Codul pentru acest model și primul test vor fi lăsate mai jos.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.svm import SVC
from sklearn.model_selection import KFold, cross_val_predict
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report, ConfusionMatrixDisplay

# eliminam formatul stiintific pentru probabilitati
np.set_printoptions(suppress=True)

# incarcam datele
df = pd.read_csv('data/Teen_Mental_Health_Dataset.csv')

# eliminam attributele stress_level, anxiety_level si addiction_level
df = df.drop(['stress_level', 'anxiety_level', 'addiction_level'], axis=1)

# transformam variabile text in numerice
def encode_text(df, columns):
    le = LabelEncoder()
    for col in columns:
        df[col] = le.fit_transform(df[col])
    return df

categoriale = ['gender', 'platform_usage', 'social_interaction_level']
df = encode_text(df, categoriale)

# separare X (atribute) si y (scop)
X = df.drop('depression_label', axis=1).values.astype(np.float32)
y = df['depression_label'].values.astype(np.float32)

# standardizam datele
scaler = StandardScaler()
X_std = scaler.fit_transform(X)

# functie pentru rularea experimentului SVM
def experiment_svm(k_fold_val, kernel_val, C_val, gamma_val, prob_val,
nume_exp):
    print(f"\n\n--- {nume_exp} ---")
    print(f"Parametri: Kernel={kernel_val}, C={C_val}, Gamma={gamma_val},
Probability={prob_val}")

    # construirea modelului SVM
    svm_model = SVC(kernel=kernel_val, C=C_val, gamma=gamma_val,
probability=prob_val, random_state=1)
    kf = KFold(n_splits=k_fold_val, shuffle=True, random_state=42)

    # predictiile modelului prin k-fold
    y_pred = cross_val_predict(svm_model, X_std, y, cv=kf)

    # afisarea numarului de instante
```

```

n_test = len(y)
print("Numarul de instante noi =", n_test)

# calcularea erorii de predictie ca raport
n_pred_incorect = (y != y_pred).sum()
print("Eroarea de predictie : % 5.2f" % (n_pred_incorect / n_test))

# acuratetea de predictie
print("Acuratetea este % 5.2f" % accuracy_score(y, y_pred))

# daca am setat probability=True, afisam si probabilitatile
if prob_val:
    y_proba = cross_val_predict(svm_model, X_std, y, cv=kf,
method='predict_proba')
    print('Probabilitatile ptr. primele 5 instante test (%):\n',
y_proba[:5] * 100)

# afisare raport de clasificare
print("\nRaportul de clasificare:")
print(classification_report(y, y_pred))

# matricea de confuzie
cm = confusion_matrix(y, y_pred)
print('Matricea de confuzie:\n', cm)

# reprezentare grafica
fig, ax = plt.subplots(1, 2, figsize=(12, 5))

# matricea de confuzie grafica
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=[0,
1])
disp.plot(ax=ax[0], cmap='Purples')
ax[0].set_title(f"Matrice Confuzie {nume_exp}")

# comparare valori reale vs predictii (esantion 50 instante)
t = pd.DataFrame({'real': y[:50], 'predit': y_pred[:50]})
ax[1].plot(t['real'].tolist(), label='Real', marker='o', linestyle='')
ax[1].plot(t['predit'].tolist(), label='Predit', marker='x',
linestyle='')
ax[1].set_title("Real vs Predictie SVM")
ax[1].legend()

plt.tight_layout()
plt.show()

# experiment 1
experiment_svm(k_fold_val=5, kernel_val='linear', C_val=1.0,
gamma_val='scale', prob_val=True, nume_exp="Exp SVM 1")

```

În continuare va fi prezentat mai jos tabelul centralizator în care vor fi scrise următoarele valori: k (Folds), kernel, gamma, C, acuratețea medie și precizia (pentru valori de 0 și valori de 1). Totodată vor fi puse mai jos 2 poze cu matricea de confuzie, respectiv compararea valorilor reale cu predicțiile.

Nr. exp.	K	Kernel	Gamma	C	Prob.	Acuratețe medie	Precizie (0 / 1)
1	5	linear	scale	1.0	True	0.97	0.97 / 0.0
2	5	rbf	scale	1.0	True	0.97	0.97 / 0.0
3	5	rbf	scale	10	True	0.96	0.98 / 0.12
4	5	poly	scale	1.0	True	0.97	0.97 / 0.0
5	10	rbf	auto	1.0	True	0.97	0.97 / 0.0

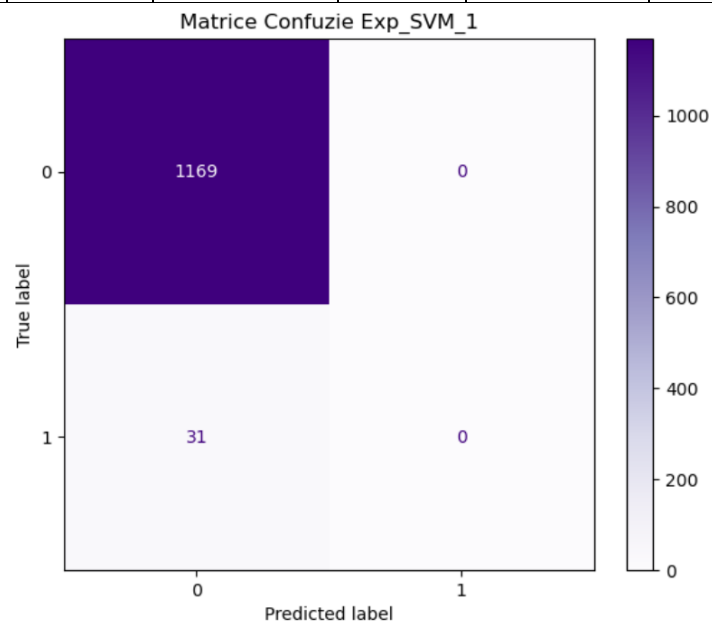


Fig. 7 - Matricea de confuzie exp. 1 Support Vector Machine

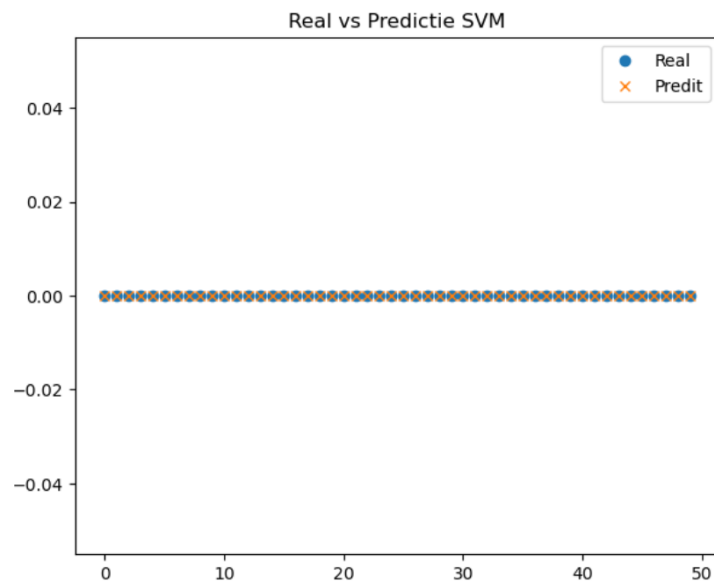


Fig. 8 - Valori reale vs predicție exp. 1 Support Vector Machine

Arbori de decizie (cod și experimente)

Codul pentru acest model și primul test vor fi lăsate mai jos.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, export_text, plot_tree
from sklearn.model_selection import KFold, cross_val_predict
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report, ConfusionMatrixDisplay
from sklearn import metrics

# incarcarea bazei de date
df = pd.read_csv('data/Teen_Mental_Health_Dataset.csv')

# eliminam attributele stress_level, anxiety_level si addiction_level
df = df.drop(['stress_level', 'anxiety_level', 'addiction_level'], axis=1)

# transformam variabile text in numerice
def encode_text(df, columns):
    le = LabelEncoder()
    for col in columns:
        df[col] = le.fit_transform(df[col])
    return df

nume_atribute = ['age', 'gender', 'daily_social_media_hours',
'platform_usage',
                'sleep_hours', 'screen_time_before_sleep',
'academic_performance',
                'physical_activity', 'social_interaction_level']

df = encode_text(df, ['gender', 'platform_usage',
'social_interaction_level'])

# separare X (atribute) si y (scop)
X = df[nume_atribute].values
y = df['depression_label'].values

# functie pentru rularea experimentului
def experiment_arbore(k_fold_val, criteriu, adancime, min_leaf, nume_exp):
    print(f"\n\n--- {nume_exp} (Criterion={criteriu}, MaxDepth={adancime},
MinLeaf={min_leaf}) ---")

    # crearea modelului arborelui de decizie
    dt = DecisionTreeClassifier(criterion=criteriu, max_depth=adancime,
                               min_samples_leaf=min_leaf, random_state=1)

    kf = KFold(n_splits=k_fold_val, shuffle=True, random_state=42)

    # realizarea clasificarii prin k-fold
    y_pred = cross_val_predict(dt, X, y, cv=kf)

    # calcularea acuratetii modelului si afisarea acesteia
    print("Accuracy:", metrics.accuracy_score(y, y_pred))
```



```

# afisare raport de clasificare
print(classification_report(y, y_pred))

# generare matrice de confuzie
cm = confusion_matrix(y, y_pred)
print("Matricea de confuzie:\n", cm)

# antrenam modelul pe tot setul pentru a genera regulile si graficul
dt.fit(X, y)

# afisarea arborelui de decizie format text
text_representation = export_text(dt, feature_names=nume_atribute)
print("\nStructura arborelui (Reguli):")
print(text_representation)

# reprezentare grafica
fig = plt.figure(figsize=(20,15))
_ = plot_tree(dt,
              feature_names=nume_atribute,
              class_names=['No Depression', 'Depression'],
              filled=True,
              fontsize=10)
plt.title(f"Arbore de Decizie - {nume_exp}")
plt.show()

# experiment 1
experiment_arbore(k_fold_val=5, criteriu='gini', adancime=3, min_leaf=5,
nume_exp="Arbore Gini")

```

În continuare va fi prezentat mai jos tabelul centralizator în care vor fi scrise următoarele valori: k (Folds), criteriu, max_depth, min_samples_lead, acuratețea medie și precizia (pentru valori de 0 și valori de 1). Totodată vor fi puse mai jos 2 poze cu afișarea arborelui de decizie în format text, respectiv reprezentarea grafică a acestuia.

Nr. exp.	K	Criterion	Max_depth	Min_samples_leaf	Acuratețe medie	Precizie (0 / 1)
1	5	gini	3	5	0.9725	0.98 / 0.33
2	5	entropy	5	5	0.9691	0.97 / 0.0
3	5	gini	None	2	0.9608	0.98 / 0.21
4	5	gini	4	20	0.9741	0.97 / 0.0

```

Structura arborelui (Reguli):
|--- sleep_hours <= 5.85
|   |--- daily_social_media_hours <= 5.05
|   |   |--- class: 0
|   |   |--- daily_social_media_hours > 5.05
|   |       |--- sleep_hours <= 4.65
|   |       |   |--- class: 0
|   |       |   |--- sleep_hours > 4.65
|   |       |       |--- class: 0
|--- sleep_hours > 5.85
|   |--- class: 0

```

Fig. 9 – Arborele de decizie în format text exp. 1 Arbori de decizie

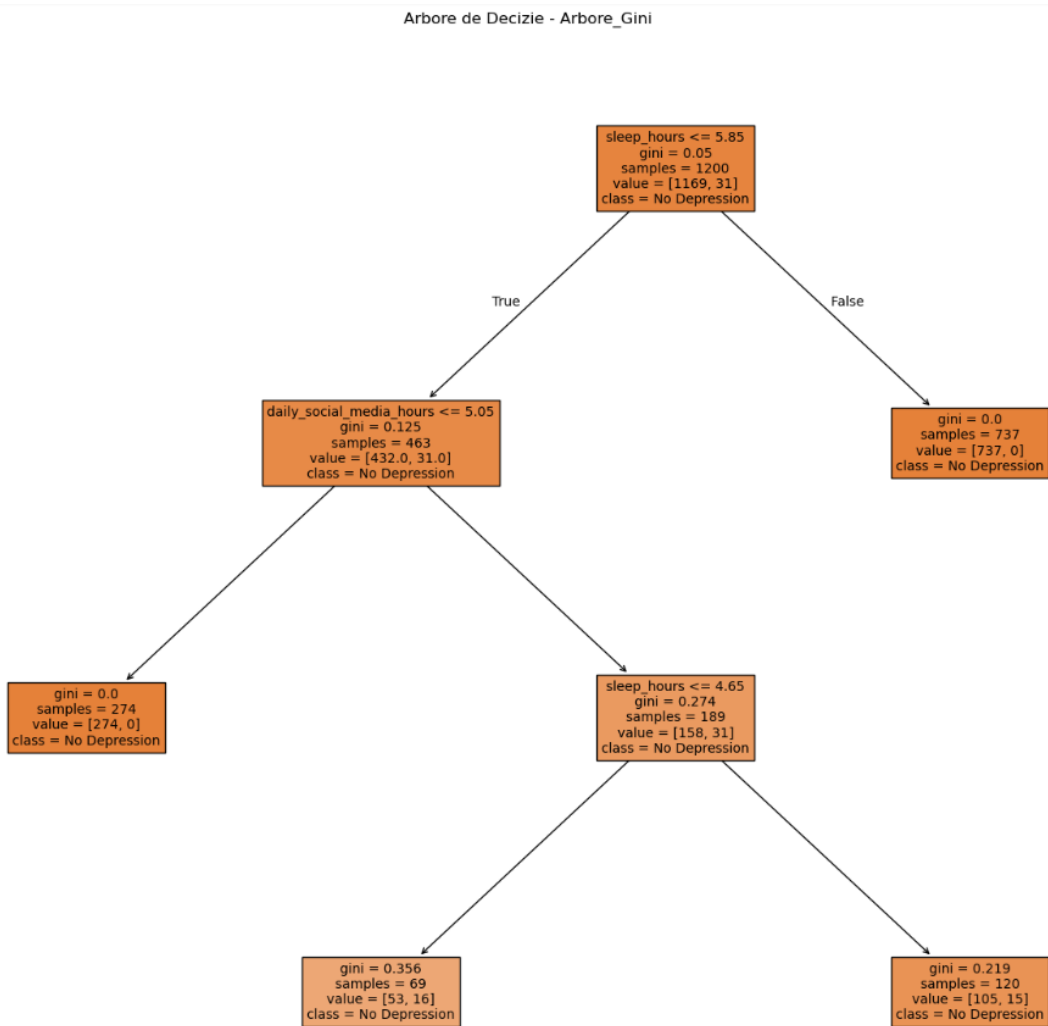


Fig. 10 - Arborele de decizie reprezentat grafic exp. 1 Arbori de decizie

Regulile de decizie

- Dacă **sleep_hours** ≤ 5.85 și **daily_social_media_hours** ≤ 5.05 ATUNCI clasa e 0 (nu are depresie)
- Dacă **sleep_hours** ≤ 5.85 și **daily_social_media_hours** > 5.05 și **academic_performance** ≤ 3.79 și **daily_social_media_hours** > 7.15 și **academic_performance** ≤ 2.32 ATUNCI clasa e 1 (are depresie)
- ATUNCI clasa e 0 (nu are depresie)
- Dacă **sleep_hours** ≤ 5.85 și **daily_social_media_hours** > 5.05 și **screen_time_before_sleep** ≤ 0.55 ATUNCI clasa e 1 (are depresie)
- Dacă **sleep_hours** ≤ 5.85 și **daily_social_media_hours** > 5.05 și **sleep_hours** ≤ 4.65 și **age** ≤ 16.50 ATUNCI clasa e 0 (nu are depresie)

Tabel centralizator comparativ

Conform tabelelor de mai sus se poate obține următorul tabel centralizator din care se pot extrage concluzia de mai jos.

Model	Nr. exp.	Parametri cheie	Acuratețe medie	Precizie clasa 0 (sănătos)	Precizie clasa 1 (depresie)
Perceptron	4	k=5, eta0=0.01, early_stop=True	0.9575	0.97	0.00
Naive Bayes	2	k=10, GaussianNB	0.9700	0.97	0.00
KNN	1	k=5, n_neighbors=5, p=2	0.9700	0.97	0.00
SVM	1	k=5, kernel=linear, C=1.0	0.9700	0.97	0.00
Arbori decizie	4	k=5, gini, max_depth=4, leaf=20	0.9741	0.97	0.00

Arborele de decizie a obținut cea mai mare valoare a acurateței medii (**0.9741**), depășind ușor modelele SVM și Naive Bayes.